

PROGETTO FINE CORSO JAVA 2025

Realizzare un'applicazione **backend REST** in Java con Spring Boot per la gestione di una scuola privata che offre **corsi post-diploma** in ambito tecnico (es. programmazione, grafica, networking). L'applicazione dovrà gestire in modo efficace studenti, docenti, corsi offerti, iscrizioni ai corsi e valutazioni finali.

L'interfaccia sarà **esclusivamente backend**, testabile tramite Postman e completamente documentata con Swagger/OpenAPI. La **progettazione del dominio (entità, relazioni e regole)** sarà **vostra responsabilità** e costituisce parte fondamentale del lavoro.

REQUISITI FUNZIONALI ESSENZIALI

- L'applicazione deve prevedere operazioni CRUD di base e avanzate per le entità del sistema
- Il progetto deve contenere almeno **10 entità distinte**, di cui **almeno 2** collegate da una relazione @ManyToMany
- Dataset pre-caricato all'avvio dell'applicazione contenente dati realistici
- Esportazione in formato CSV delle anagrafiche (es. studenti, docenti o iscrizioni)
- Generazione di un file PDF con il profilo completo di uno studente
- Esportazione in formato Excel con l'elenco dei percorsi di studio e dei relativi esami
- Endpoint di registrazione di un nuovo utente al sistema

Punti opzionali avanzati (facoltativi ma consigliati)

- *Paginazione e ordinamento*: gestione di liste con Pageable, parametri di ordinamento e filtri dinamici
- Task pianificati (@Scheduled): esecuzione automatica di controlli o aggiornamenti a intervalli regolari
- Query avanzate: uso di @Query, @EntityGraph o query native per logiche personalizzate complesse
- Servizi asincroni (@Async): esecuzione di logiche non bloccanti, es. post-registrazione
- Invio email (reale o simulato): integrazione con servizi email (es. Mailtrap) o mock interni
- Statistiche aggregate: endpoint dedicati con medie, conteggi, ranking o andamento
- Dockerizzazione del progetto: creazione di un Dockerfile funzionante per l'esecuzione completa dell'app
- Uso di Mockito nei test unitari: mocking di dipendenze nei test con Mockito

REQUISITI TECNICI

- Java 21 o superiore
- Utilizzo dell'ecosistema Spring studiato durante il percorso
- Database relazionale a scelta
- Framework di logging a scelta
- API documentate con Swagger/OpenAPI
- Test unitari tramite JUnit
- Coerenza e validità dei dati gestite a livello logico e strutturale

REQUISITI DI PROGETTAZIONE

- Architettura e organizzazione secondo il pattern MVC
- Separazione tra entità e DTO
- Gestione centralizzata delle eccezioni
- Logging nei punti chiave del flusso applicativo
- Endpoint RESTful coerenti
- Utilizzo dei Design Pattern dove necessario, con commenti esplicativi
- Qualità del codice e rispetto delle best practice Java

CRITERI DI VALUTAZIONE E CONSEGNA

- Repository GitHub completo del progetto
- File `.gitignore` configurato
- File `README.md` con le sezioni richieste (in allegato)
- File Postman `.json` nella cartella `/postman`

La consegna del progetto è prevista entro **sabato 27 aprile 2025 alle ore 18.00**.

Farà **fede l'orario dell'ultimo commit Git** contenente il commento esatto:

`"VERSIONE FINALE PROGETTO FINE CORSO JAVA"`

In caso di ritardo nella consegna, sarà applicata una **penalità di 2 punti per ogni giorno di ritardo**, conteggiata a partire dalle ore 18.01 del giorno di scadenza.

Di seguito i punti rilevanti per i criteri di valutazione:

CRITERIO	DESCRIZIONE
Modellazione del dominio	Almeno 10 entità con relazioni corrette, incluse due @ManyToMany
Architettura e organizzazione	Uso del pattern MVC, separazione dei componenti
Uso di DTO	Separazione tra entità e oggetti di trasferimento dati
CRUD e logiche avanzate	Endpoint completi e funzionali, con logiche di filtro o ricerca
Funzionalità	CSV, PDF, Excel implementati e funzionanti
Autenticazione JWT	Login, registrazione e protezione degli endpoint
Logging e gestione errori	Logging significativo, gestione centralizzata delle eccezioni
Documentazione API	Swagger/OpenAPI con descrizione degli endpoint
Repository GitHub	README completo, struttura pulita, commit descrittivi
Test con JUnit	Presenza di test unitari su componenti della logica con coverage > 50%
Qualità del codice e best practice	Codice leggibile, coerente, ben strutturato e aderente agli standard Java